

中国科学技术大学实验报告

课程名称：系统与amp;控制实验 II

实验名称：水箱液位控制系统设计与实现

姓名学号：

2026 年 6 月 12 日

目录

1 实验目的	1
2 实验设备与系统	1
3 实验内容与核心原理	1
3.1 控制对象的建模	1
3.1.1 实验目的	1
3.1.2 实验原理	1
3.1.3 实验内容	2
3.2 数字 PID 控制参数工程整定	3
3.2.1 实验目的	3
3.2.2 实验原理	3
3.2.3 实验内容	3
3.3 Smith 时延补偿器设计	5
3.3.1 实验目的	5
3.3.2 实验内容	5
3.3.3 Smith 时延补偿器原理	5
3.4 大林控制器算法 (Dahlin Algorithm)	7
3.4.1 实验目的	7
3.4.2 大林控制器设计步骤	7
3.4.3 期望闭环时间常数 $\tau = 15$ 时的计算过程	8
3.4.4 期望闭环时间常数 $\tau = 40$ 时的计算过程	8
3.4.5 控制参数对比与分析	8
3.5 卡尔曼控制器设计	9
3.5.1 实验目的	9
3.5.2 实验内容	9
3.5.3 卡尔曼控制器 $D(z)$ 表达式计算过程	10

目录	II
3.6 自主设计：基于内模控制（IMC）的水箱液位系统	12
3.6.1 IMC 在实际生产中的应用	12
3.6.2 内模控制（IMC）核心原理与离散化推导	12
3.6.3 控制效果展示与结果分析	13
4 实验总结	15
A 系统建模代码	16
B Smith 补偿器代码	16
C 大林控制器代码	17
D 卡尔曼控制器代码	18
E IMC 控制器代码	19

1 实验目的

1. 深入理解计算机控制系统设计的基本理论、方法及其应用。
2. 掌握常用控制器的设计与调试方法，包括 PID 控制、Smith 预估器、大林 (Dahlin) 算法以及卡尔曼 (Kalman) 算法等实用控制技术。
3. 熟悉计算机实时控制系统的基本结构，增强对控制系统硬件组成和软件实现过程的感性认识。
4. 熟悉 MATLAB 仿真软件的使用，理解 PID 控制器的工作原理及设计方法。
5. 能够独立设计和实现 Smith 预估器、大林控制器、卡尔曼控制器等先进控制策略，并将其应用于实际工程控制问题中，提高分析和解决工程问题的能力。

2 实验设备与系统

- 实验选用水箱液位控制系统作为实验对象，该系统主要由水箱、压力传感器、控制箱、计算机和 I/O 接口等环节组成。
- 水箱作为控制对象，其液位作为被控量。水箱液位的变动被转换成 **1-5V** 的标准电信号，由 I/O 接口的 A/D 转换成二进制的数字信号后，送入计算机存储。
- 计算机按照设定的控制算法计算出下一步的控制量，经 I/O 接口中的 D/A 转换成 **4-20mA** 标准模拟信号控制水泵加水，从而实现对水箱液位的反馈控制。

3 实验内容与核心原理

3.1 控制对象的建模

3.1.1 实验目的

建立实验控制系统的模型。

3.1.2 实验原理

水箱控制系统可近似为一个一阶纯滞后系统。

- **模型表达式：** $G(S) = \frac{K}{QS + 1} e^{-\tau S}$ 。
- **参数含义：** τ 为系统时延，增益 $K = \Delta T / \Delta U$ (ΔT 为液位差， ΔU 为开环输入值差)。只考虑一阶惯性环节时，时间常数 Q 满足 $y^*(Q) = 1 - e^{-1} = 0.63$ 。
- **系统离散化参数：** 统一下发设定采样周期参数为 $T=1s$ ，以确保系统极点分析与 Bode 图绘制的精度和一致性。

3.1.3 实验内容

熟悉使用水箱控制系统软件，通过测试水箱控制系统的开环阶跃响应曲线，并由响应曲线获取水箱液位控制系统开环动态特性参数，及时间参数 T ，时延 τ ，稳态增益 K 。利用设计的水箱控制系统软件做出图 1：

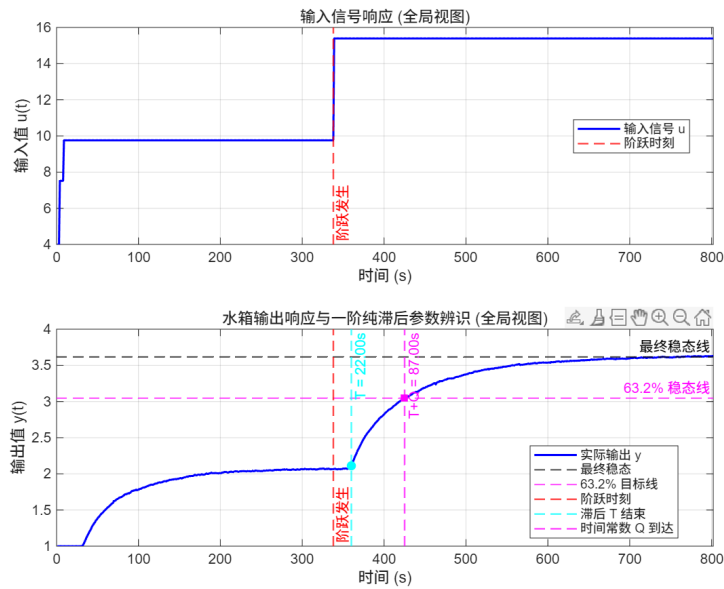


图 1: 系统阶跃响应的辨识结果

依据此辨识结果以及代码运行结果，得出系统模型为：

$$G(S) = \frac{0.2750}{65S + 1} e^{-22S} \quad (1)$$

(系统建模完整代码详见附录 A)

3.2 数字 PID 控制参数工程整定

3.2.1 实验目的

1. 建立对数字 PID 控制器及其性能的感性认识；了解数字 PID 控制器参数（采样周期，比例系数 K_p ，积分时间 T_i ，微分时间 T_d ）对控制系统的影响。
2. 学习数字 PID 控制器参数常用的工程整定方法。
3. 掌握数字 PID 控制器的设计方法。

3.2.2 实验原理

利用 PID 控制器对系统的反馈信号进行处理，使其达到期望状态。

- **Z 传递函数：** $D(z) = K_P + K_I \frac{1}{1 - z^{-1}} + K_D(1 - z^{-1})$ 。
- **位置式差分方程：** $u(k) = K_P e(k) + K_I \sum_{i=0}^k e(i) + K_D[e(k) - e(k-1)]$ 。
- **参数整定：** 采用扩充响应曲线法整定控制率参数，根据系统要求选取控制度（如 1.05 或 1.2）查找相应的 K_P 、 T_i 、 T_d 系数比值。
- **算法优化：** 实际工程中常采用积分分离、梯形积分或抗积分饱和等优化 PID 算法以改善动态性能。

3.2.3 实验内容

1. 根据被控系统开环动态特性和控制性能要求选择控制率（比例积分微分的选取）。
2. 用扩充响应曲线法，整定所选的控制率中的参数。

表 1: 扩充响应曲线法 PID 参数表

控制度	控制率	T_s/τ	$K_P/(T/\tau)$	T_i/τ	T_d/τ
1.05	Pi	0.1	0.84	3.4	-
1.05	PID	0.05	1.15	2.0	0.45
1.2	Pi	0.2	0.72	3.6	-
1.2	PID	0.16	1.0	1.9	0.55
1.5	Pi	0.5	0.68	3.9	-
1.5	PID	0.34	0.85	1.62	0.65

3. 编写数字 PID 控制器设计程序并调试运行。
4. 分析实验结果：液位系统的过渡时间、超调量和稳态误差。

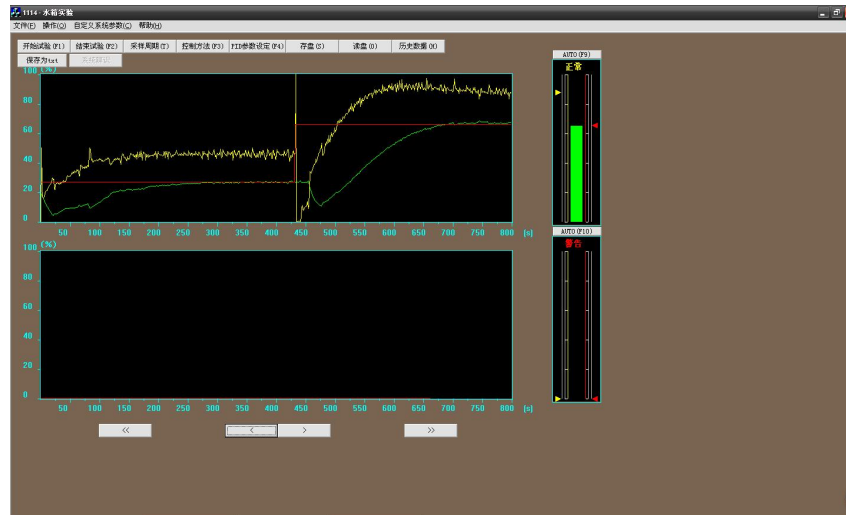


图 2: PID 运行图 1

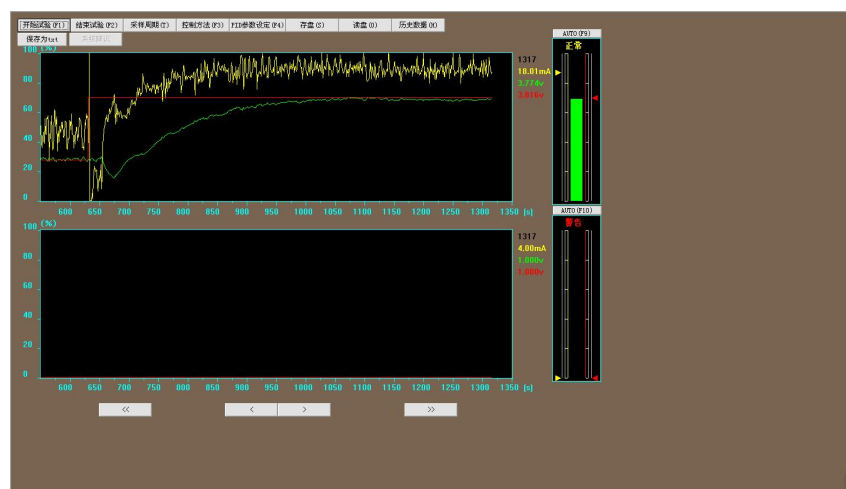


图 3: PID 运行图 2

其中采用的参数分别为：

pid1: $K_p = 3.39$, $K_i = 0.077$, $K_d = 33.56$

pid2: $K_p = 6.8$, $K_i = 0.077$, $K_d = 33.56$

3.3 Smith 时延补偿器设计

3.3.1 实验目的

大多数工业系统都有或大或小的时延，若系统的难控制度（即时延 τ 和等效时间 T 之比）大于 0.5，则用常规的 PID 控制很难获得满意的控制性能。如果生产要求较高的控制性能，则必须采用较复杂的控制策略。工业上，控制大时延系统常用的一种控制策略是加 Smith 时延补偿器的 PID 控制算法。一般情况下可以得到较好的控制效果。为此，我们设计了这个实验，其目的是：

1. 认识和理解 Smith 时延补偿器控制大时延系统的机理和效果。
2. 掌握实际控制系统的 Smith 时延补偿器的设计、实现和调试方法和技术。

3.3.2 实验内容

1. 将模型进行等效离散化。
2. 基于被控对象模型，设计出系统的 Smith 时延补偿器并且记下带 Smith 时延补偿器的 PID 控制效果。

3.3.3 Smith 时延补偿器原理

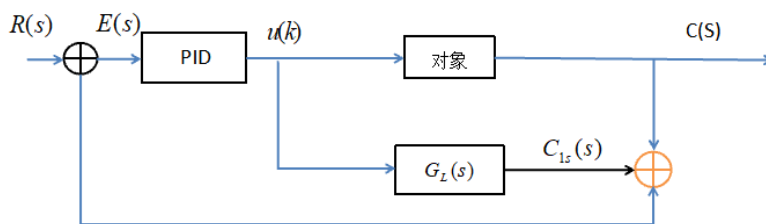


图 4: Smith 时延补偿器系统闭环方框图

当系统的难控制度（时延 τ 和等效时间 T 之比）大于 0.5 时，常规 PID 难以获得满意性能，需引入 Smith 补偿器。

- 被控对象（一阶惯性加纯滞后）：

$$G(s) = \frac{K}{\tau_1 s + 1} e^{-\theta s} \quad (2)$$

（ $\theta = lT$ 为系统时延， l 为延迟步数，采样周期 $T = 1s$ ，此处 $l = 22$ ）

- **Smith 时延补偿器：**

$$G_L(s) = \frac{K(1 - e^{-lTs})}{\tau_1 s + 1} \quad (3)$$

引入补偿器后，PID 控制器的误差输入由 $e(k) = R(k) - C(k)$ 修正为：

$$e(k) = R(k) - [C(k) + c_{ls}(k)] \quad (4)$$

($R(k)$ 为设定值， $C(k)$ 为实际反馈， $c_{ls}(k)$ 为预估补偿量)

将 $G_L(s)$ 拆分为惯性环节 $\frac{1}{\tau_1 s + 1}$ 与纯滞后差值环节进行离散化：

- **无时延中间变量 $v(k)$ 递推：**

$$v(k) = (1 - Q)v(k - 1) + Qu(k) \quad (5)$$

(平滑系数 $Q = \frac{T}{T + \tau_1}$ ， $u(k)$ 为控制器输出量)

- **补偿器最终离散输出 $c_{ls}(k)$ ：**

$$c_{ls}(k) = K[v(k) - v(k - l)] \quad (6)$$

我们选取 $K = 0.275$ ，运行得到下面结果图 5，对比 PID1 运行结果发现效果明显改善。使用 Smith 时延补偿器后，系统的超调量明显减小，过渡时间也明显缩短，因此 Smith 时延补偿器对系统的控制效果有明显的改善作用。

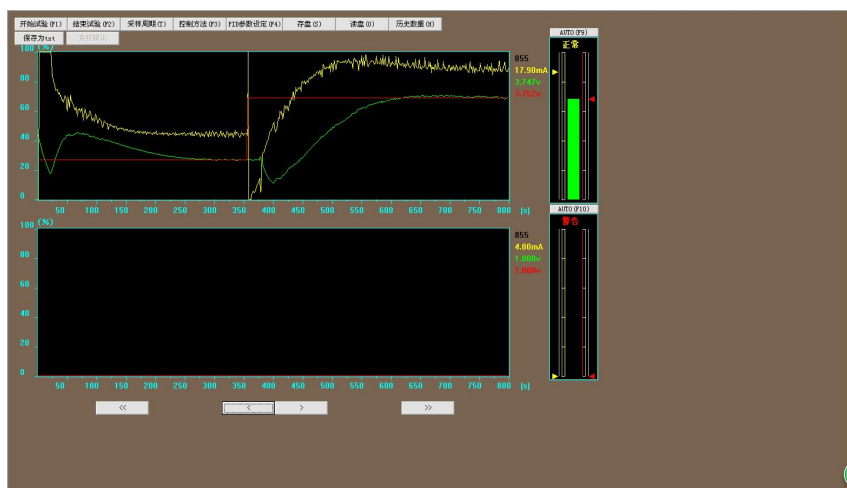


图 5: Smith 补偿器运行结果图

3.4 大林控制器算法 (Dahlin Algorithm)

3.4.1 实验目的

1. 认识和理解大林控制算法控制大时延系统的机理和效果。
2. 掌握实际控制系统的大林算法的设计和实现。

3.4.2 大林控制器设计步骤

- 设计闭环传递函数 $W(s) = Z\{G_h(s)W_p(s)\}$, 其中 $W_p(s) = \frac{e^{-\theta s}}{\tau s + 1} = \frac{e^{-T_s s}}{\tau s + 1}$ 。
- 做 Z 变换得到 $W(z) = z^{-(l+1)} \frac{1 - e^{-T/\tau}}{1 - e^{-T/\tau} z^{-1}}$ 。
- 又因为 $W(z) = \frac{D(z)G_h G(z)}{1 + D(z)G_h G(z)}$, 且 $G_h G(z) = K \frac{1 - e^{-T/\tau_1}}{1 - e^{-T/\tau_1} z^{-1}} z^{-(l+1)}$, 故大林控制器:

$$D(z) = \frac{W(z)}{[1 - W(z)]G_h G(z)} = \frac{1 - e^{-T/\tau}}{K(1 - e^{-T/\tau_1})} \cdot \frac{1 - e^{-T/\tau_1} z^{-1}}{1 - e^{-T/\tau} z^{-1} - (1 - e^{-T/\tau}) z^{-(l+1)}} \quad (7)$$

- 其一般形式为 $D(z) = \frac{a(1 - bz^{-1})}{1 - cz^{-1} - (1 - c)z^{-(l+1)}}$, 其中:
 $a = \frac{1 - e^{-T/\tau}}{K(1 - e^{-T/\tau_1})}$, $b = e^{-T/\tau_1}$, $c = e^{-T/\tau}$ 。
- 因此, 大林控制器的差分方程为 $u(k) = ae(k) - abe(k-1) + cu(k-1) + (1-c)u(k-l-1)$ 。程序实现时需维护历史控制量数组以提取滞后项。

被控对象传递函数为:

$$G_p(s) = \frac{0.2750e^{-22s}}{65S + 1}$$

将被控对象进行离散化得到:

采样周期 $T = 1s$

$$G(z) = Z \left[\frac{1 - e^{-Ts}}{s} \cdot \frac{0.2750e^{-22s}}{65S + 1} \right]$$

提取基本参数:

- 采样周期: $T = 1s$
- 系统放大倍数: $K = 0.2750$

- 系统时间常数: $\tau_1 = 65$
- 纯滞后步数: $l = 22$

3.4.3 期望闭环时间常数 $\tau = 15$ 时的计算过程

(旨在追求较快的系统响应速度)

1. 计算参数 b (与系统自身惯性相关, 为固定值): $b = e^{-T/\tau_1} = e^{-1/65} \approx 0.9847$
2. 计算参数 c (与期望响应速度相关): $c = e^{-T/\tau} = e^{-1/15} \approx e^{-0.0667} \approx 0.9355$
3. 计算控制器的比例增益基准 a : $a = \frac{1-c}{K(1-b)} = \frac{1-0.9355}{0.2750 \times (1-0.9847)} \approx 15.3614$
4. 确定控制器的差分方程 (交叉项 $a \cdot b \approx 15.1268$, $1-c \approx 0.0645$):

$$u(k) = 15.3614e(k) - 15.1268e(k-1) + 0.9355u(k-1) + 0.0645u(k-23) \quad (8)$$

3.4.4 期望闭环时间常数 $\tau = 40$ 时的计算过程

(旨在追求极其平稳、不超调的系统响应)

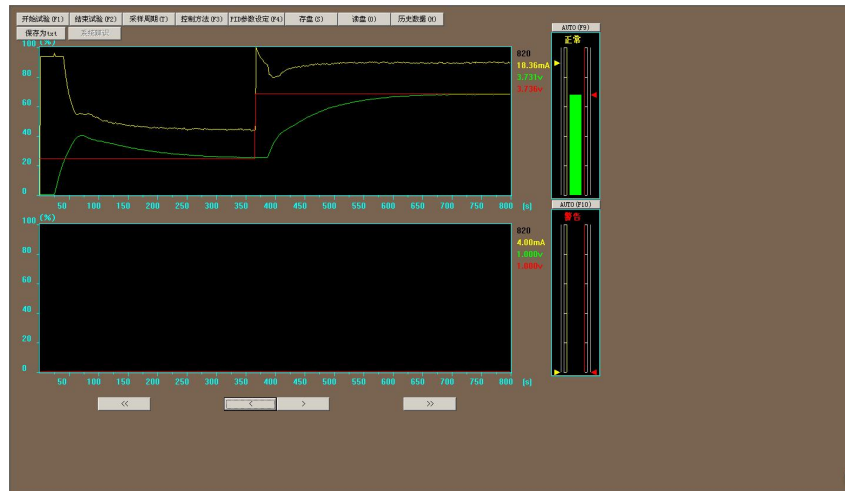
1. 计算参数 b : $b \approx 0.9847$
2. 计算参数 c : $c = e^{-1/40} = e^{-0.025} \approx 0.9753$
3. 计算比例增益基准 a : $a = \frac{1-0.9753}{0.2750 \times (1-0.9847)} \approx 5.8808$
4. 确定控制器的差分方程 (交叉项 $a \cdot b \approx 5.7911$, $1-c \approx 0.0247$):

$$u(k) = 5.8808e(k) - 5.7911e(k-1) + 0.9753u(k-1) + 0.0247u(k-23) \quad (9)$$

代码实现部分参见附录 C。

3.4.5 控制参数对比与分析

- 当 $\tau = 15$ 时, 由于期望的闭环响应时间较短, 控制器的比例增益部分 ($a \approx 15.36$) 较大, 控制作用更猛烈, 能促使系统快速达到设定值。
- 当 $\tau = 40$ 时, 期望闭环响应更平缓保守, 控制器的比例增益 ($a \approx 5.88$) 显著减小, 使得控制输出更加温和, 能有效避免执行机构过载或引发系统振荡。

图 6: $\tau = 15$ 时大林控制效果图 7: $\tau = 40$ 时大林控制效果

3.5 卡尔曼控制器设计

3.5.1 实验目的

1. 认识和理解卡尔曼控制算法控制大时延系统的机理和效果。
2. 掌握实际控制系统的卡尔曼算法的设计和实现。

3.5.2 实验内容

2. 将模型进行等效离散化。

3. 基于被控对象等效离散化模型设计卡尔曼控制算法，编写出实现程序，将其嵌入实验软件中。
4. 将设计的卡尔曼算法投入运行，并经过调试获得预期控制性能。

卡尔曼控制器是一种最少拍控制策略，期望闭环系统在有限个采样周期内达到稳态。

- 将被控对象离散化表达为 $G(z) = \frac{A(z^{-1})}{B(z^{-1})}z^{-d}$ 。
- 设计目标是使系统对设定的阶跃输入实现无静差跟踪，期望闭环传递函数 $\Phi(z) = z^{-d}$ 。
- 控制器传递函数 $D(z) = \frac{\Phi(z)}{G(z)(1 - \Phi(z))}$ ，代入模型可得控制器差分方程。

3.5.3 卡尔曼控制器 $D(z)$ 表达式计算过程

1. 已知系统参数与离散化模型

$$G(s) = \frac{0.2750}{65s + 1}e^{-22s}$$

结合代码中设定的纯滞后步数 $L = 22$ ，可知系统采用的采样周期为 $T = 1s$ 。提取关键参数：

- 放大倍数 $K = 0.2750$
- 时间常数 $\tau_1 = 65$
- 纯滞后步数 $l = 22$

由带有零阶保持器的等效离散化公式，求对象的 Z 传递函数：

$$G(z) = \mathcal{Z} \left[\frac{1 - e^{-Ts}}{s} G(s) \right] = \frac{K(1 - e^{-T/\tau_1})z^{-(l+1)}}{1 - e^{-T/\tau_1}z^{-1}}$$

代入具体数值进行计算：

- $e^{-T/\tau_1} = e^{-1/65} \approx 0.9847$
- $K(1 - e^{-T/\tau_1}) = 0.2750 \times (1 - 0.9847) \approx 0.004198$

得到系统实际的离散传递函数为：

$$G(z) = \frac{0.004198z^{-23}}{1 - 0.9847z^{-1}} \quad (10)$$

2. 卡尔曼控制器 $D(z)$ 的设计对应标准形式 $G(z) = \frac{kM(z)}{N(z)}$ 中，可得：

- $k = 0.004198$
- $M(z) = z^{-23}$ ，因此 $M(1) = 1^{-23} = 1$
- $N(z) = 1 - 0.9847z^{-1}$

代入最小拍无波纹控制器表达式：

$$D(z) = \frac{N(z)}{k[M(1) - M(z)]} = \frac{1 - 0.9847z^{-1}}{0.004198(1 - z^{-23})} \quad (11)$$

3. 最终表达式将常数项除开以化简表达式 ($1/0.004198 \approx 238.2$)，得到最终的 $D(z)$ 表达式：

$$D(z) = \frac{238.2(1 - 0.9847z^{-1})}{1 - z^{-23}} = \frac{238.2 - 234.6z^{-1}}{1 - z^{-23}} \quad (12)$$

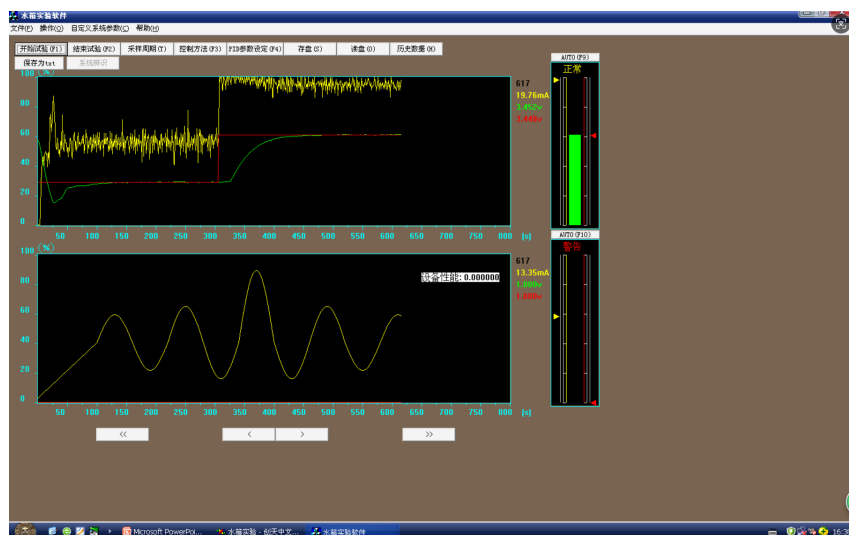


图 8: 卡尔曼控制器效果图

代码部分详见附录 D。

3.6 自主设计：基于内模控制（IMC）的水箱液位系统

3.6.1 IMC 在实际生产中的应用

内模控制（Internal Model Control, IMC）自 1982 年由 Garcia 和 Morari 提出以来，已在石化、精馏、造纸、冶金等流程工业中得到广泛应用。其核心优势在于：将过程模型显式嵌入控制结构，使控制器设计直接依赖于模型的可逆部分，并通过低通滤波器实现鲁棒性与响应速度的单参数整定。例如，在乙烯裂解炉温度控制、蒸馏塔组分控制等大纯滞后、强非线性场景中，IMC 能够有效抑制模型失配和测量噪声，保持闭环稳定。相比于传统 PID 的多参数试凑，IMC 的滤波器时间常数 λ 物理意义明确，便于现场工程师根据工艺要求（如允许的最大超调、调节时间）进行快速整定，因此成为先进过程控制中的常用策略之一。

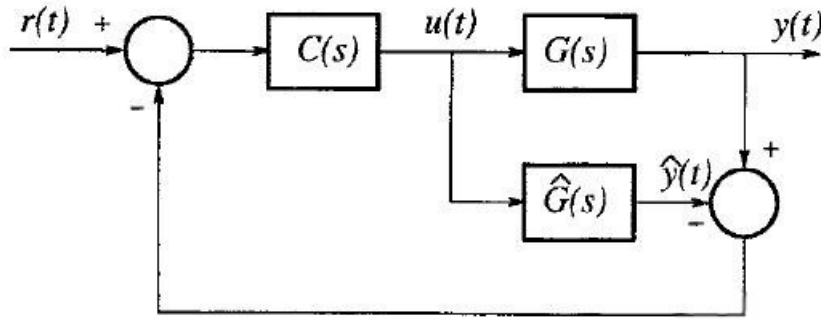


图 9: 典型 IMC 系统框图

3.6.2 内模控制（IMC）核心原理与离散化推导

针对实验测得的水箱液位一阶纯滞后连续模型：

$$G(s) = \frac{0.2750}{65s + 1} e^{-22s}$$

在统一的采样周期 $T = 1\text{s}$ 下，引入零阶保持器进行等效离散化，系统的纯滞后步数 $l = 22$ 。其离散 Z 传递函数为：

$$G(z) = \frac{K(1 - e^{-T/\tau_1})z^{-(l+1)}}{1 - e^{-T/\tau_1}z^{-1}} = \frac{0.004198 z^{-23}}{1 - 0.9847 z^{-1}}$$

内模控制的核心思想是将对象模型 $G(z)$ 分解为前向不包含时延及右半平面零点的“可逆部分” $G_-(z)$ 和包含全部纯时延的“不可逆部分” $G_+(z)$ ：

$$G_+(z) = z^{-23}, \quad G_-(z) = \frac{0.004198}{1 - 0.9847 z^{-1}}$$

理想的内模控制器为可逆部分的逆 $G^{-1}(z)$ 。为确保实际控制器的物理可实现性，并降低系统对模型误差和测量噪声的敏感度，串联一阶离散低通滤波器 $f(z)$ ：

$$f(z) = \frac{1 - \alpha}{1 - \alpha z^{-1}}, \quad \alpha = e^{-T/\lambda}$$

其中 λ 称为滤波时间常数，是整个控制系统中唯一需要人工整定的控制参数。

由此，最终构建的离散内模控制器 $C(z)$ 表达式为：

$$C(z) = f(z)G^{-1}(z) = \frac{1 - \alpha}{1 - \alpha z^{-1}} \cdot \frac{1 - 0.9847z^{-1}}{0.004198} = \frac{1 - \alpha}{0.004198} \cdot \frac{1 - 0.9847z^{-1}}{1 - \alpha z^{-1}}$$

定义控制器的误差输入为内模反馈误差 $e_{\text{imc}}(k) = \text{SetDataOne} - (\text{dataOne} - y_{\text{model}})$ ，控制输出增量为 $u_{\text{dev}}(k) = u(k) - 4.0$ 。将 $C(z) = \frac{u_{\text{dev}}(z)}{e_{\text{imc}}(z)}$ 展开并转化为计算机可执行的差分方程递推公式：

$$u_{\text{dev}}(k) = \alpha u_{\text{dev}}(k-1) + \frac{1 - \alpha}{0.004198} e_{\text{imc}}(k) - \frac{(1 - \alpha) \cdot 0.9847}{0.004198} e_{\text{imc}}(k-1)$$

代码实现部分参见附录 E。

3.6.3 控制效果展示与结果分析

分别设定滤波时间常数 $\lambda=5, 15, 40$ ，得到如下结果：

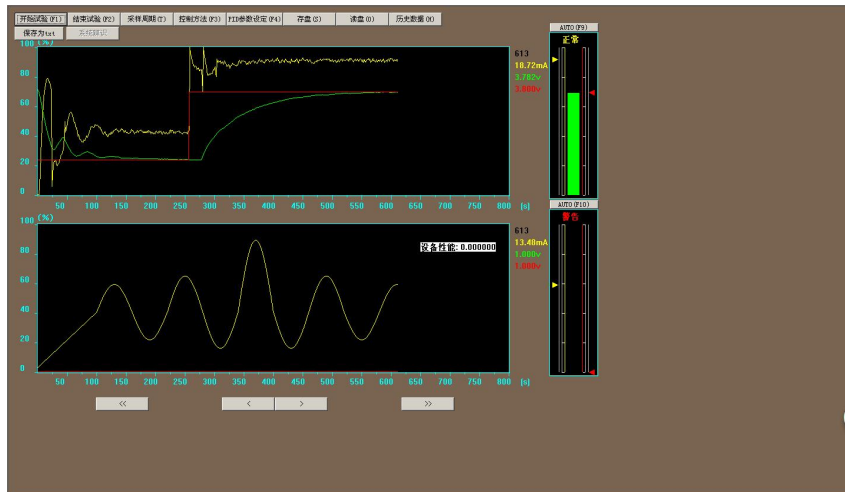
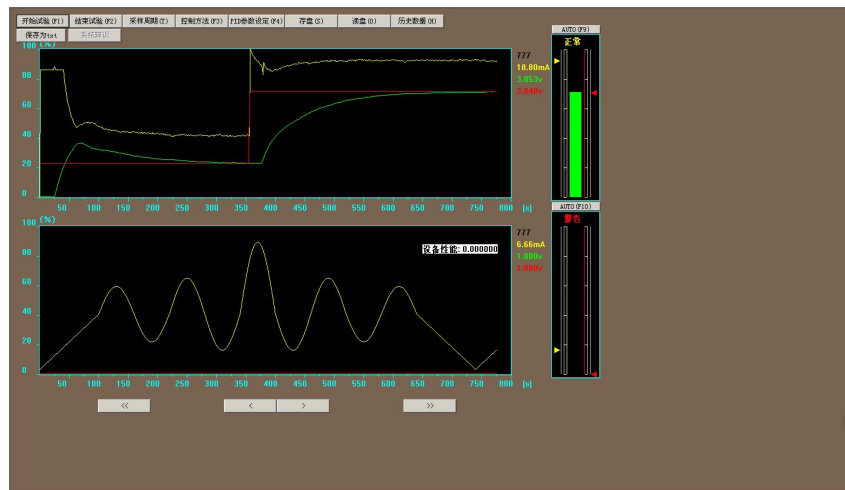
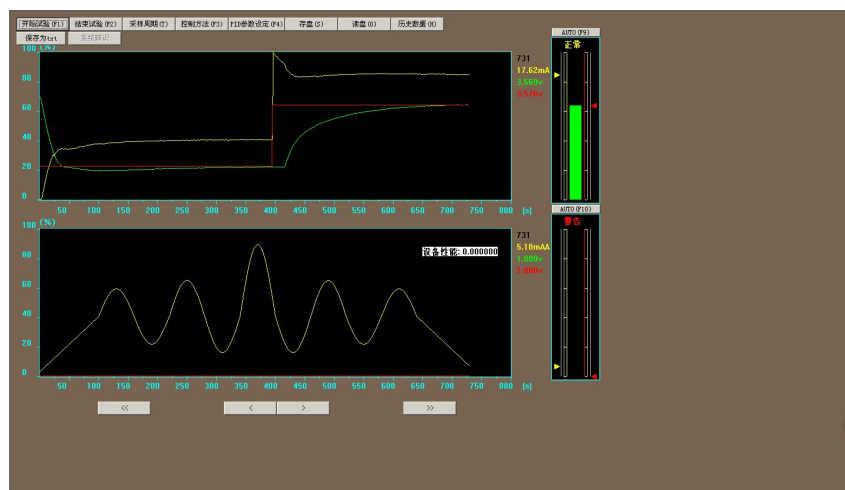


图 10: $\lambda=5$

图 11: $\lambda=15$ 图 12: $\lambda=40$

- $\lambda = 5$: 此时 $\alpha = e^{-1/5} \approx 0.8187$ 。系统响应极为迅速，上升时间较短，但液位会出现明显的超调，并伴随多次振荡。控制量输出变化剧烈，每拍增量较大。
- $\lambda = 15$: $\alpha = e^{-1/15} \approx 0.9355$ 。系统响应速度适中，上升时间与调节时间均处在合理范围，存在较小的超调量。控制量输出变化平缓，无剧烈波动。
- $\lambda = 40$: $\alpha = e^{-1/40} \approx 0.9753$ 。系统响应变得十分迟缓，上升时间和调节时间延长，但液位无超调，控制量输出变化极其温和，每拍增量很小。

4 实验总结

本次实验以水箱液位控制系统为对象，依次完成了系统模型辨识、数字 PID 参数整定、Smith 时延补偿器设计、大林控制器设计、卡尔曼（最少拍）控制器设计以及自主设计的内模控制（IMC）算法，系统掌握了大时延工业过程的多种控制策略。

1. **模型辨识**：通过开环阶跃响应实验，成功辨识出被控对象的一阶惯性加纯滞后模型 $G(s) = \frac{0.2750}{65s + 1}e^{-22s}$ 。该模型为后续控制器设计提供了准确的基础。
2. **数字 PID 控制**：采用扩充响应曲线法整定 PID 参数。实验表明，增大比例系数可加快响应但易引发超调；积分作用消除稳态误差但可能引起积分饱和；微分作用有助于改善动态性能。在大时延（ $\tau/T = 22/65 \approx 0.34$ ）条件下，常规 PID 难以同时满足快速性与稳定性要求。
3. **Smith 时延补偿器**：针对 PID 控制大时延系统的不足，引入 Smith 预估补偿结构。实验结果显示，Smith 补偿后系统超调量明显减小，过渡时间缩短，证明该策略能有效“抵消”纯滞后对闭环稳定性的不利影响。
4. **大林控制器**：根据期望闭环时间常数 $\tau = 15$ （较快）和 $\tau = 40$ （平稳）分别设计大林算法。 $\tau = 15$ 时系统响应迅速但控制量变化较剧烈； $\tau = 40$ 时响应平缓无超调。大林算法在模型精确时性能优越，但对参数变化较敏感。
5. **卡尔曼控制器**：利用离散对象模型设计最少拍无波纹控制器，其差分方程系数完全由模型参数决定。实验结果表明，卡尔曼控制器能在有限拍内实现无静差跟踪，但控制量起始时刻可能出现较大跳变，且对模型失配的鲁棒性相对较弱。
6. **自主设计——内模控制（IMC）**：作为自主设计部分，实现了 IMC 控制器，并通过单参数 λ 整定获得不同性能： $\lambda = 5$ 时响应快但有超调和振荡； $\lambda = 15$ 时综合性能良好； $\lambda = 40$ 时响应缓慢但无超调。IMC 与 Smith 预估器及大林算法在数学上存在等价关系，验证了理论一致性。

通过本次综合实验，加深了对计算机控制系统设计流程的理解，掌握了从模型辨识到多种控制器设计、编程调试的全过程。对不同控制算法在大纯滞后对象上的性能差异有了直观认识，为今后解决实际工业控制问题积累了经验。

A 系统建模代码

```

1 function [K, Q, T] = identify_FOPDT(time, u, y)
2 % 核心辨识算法：一阶惯性加纯滞后系统模型  $G(s) = K / (Q*s + 1) * \exp(-T*s)$ 
3
4 % 1. 寻找输入信号的阶跃发生时刻 (t0)
5 step_idx = find(abs(diff(u)) > 0.05 * (max(u) - min(u)), 1, 'first');
6 t0 = time(step_idx);
7
8 % 2. 计算稳态增益 K
9 u_before = mean(u(1 : step_idx-1)); % 阶跃前的初始输入
10 y_before = mean(y(1 : step_idx-1)); % 阶跃前的初始输出
11 u_after = mean(u(floor(end*0.8) : end)); % 取数据最后20%作为最终稳
    态输入
12 y_after = mean(y(floor(end*0.8) : end)); % 取数据最后20%作为最终稳
    态输出
13
14 delta_Y = y_after - y_before;
15 K = delta_Y / (u_after - u_before);
16
17 % 3. 估计纯滞后时间 T (当输出变化超过噪声阈值时视为响应开始)
18 noise_th = max(3 * std(y(1:step_idx-1)), 0.02 * abs(delta_Y));
19 start_idx = step_idx - 1 + find(abs(y(step_idx:end) - y_before) >
    noise_th, 1, 'first');
20 T = time(start_idx) - t0;
21
22 % 4. 估计时间常数 Q (到达 63.2% 稳态变化量的时间)
23 y_norm = (y - y_before) / delta_Y;
24 Q_idx = step_idx - 1 + find(y_norm(step_idx:end) >= 0.632, 1, 'first');
25 Q = time(Q_idx) - t0 - T;
26
27 % 打印最终结果
28 fprintf('核心辨识结果:  $G(s) = %.4f / (0.2f*s + 1) * \exp(-0.2f*s)\backslash n$ ', K,
    Q, T);
29 end

```

B Smith 补偿器代码

```

1 float CMgvView::smithcontrol1(float dataone, float SetDataone, float
  inDataTwo, float SetDataTwo, double Kp, double Ki, double Kd)
2 {
3     double Ke = 0.3099;
4     int Te = 25;
5     float out;
6     int Ts = 1;
7     int l = 25;
8     float q = float(Ts / (Ts + Te));
9     double C1s = Ke * (v[1] - v[0]);
10
11     out = (float)pidcontrol1(dataone + C1s, SetDataone, inDataTwo,
  SetDataTwo, Kp, Ki, Kd);
12
13     for (int i = 0; i < l; i++)
14     {
15         v[i] = v[i + 1];
16     }
17     v[l] = (1 - q) * v[l - 1] + q * out;
18
19     if (out > float(m_mvuplimit1) / 1000)
20         out = float(m_mvuplimit1) / 1000;
21     if (out < float(m_mvdownlimit1) / 1000)
22         out = float(m_mvdownlimit1) / 1000;
23
24     return out;
25 }

```

C 大林控制器代码

```

1 float CMyView::dmcccontrol1(float dataOne, float SetDataOne, float
  inDataTwo, float SetDataTwo, double Kp, double Ki, double Kd)
2 {
3     static float e2=0, e3=0;
4     static float u1=4, u2=4, eu=0;
5     static float v[65535];
6     static int L=0, i=0;
7     static float a=0, b=0, c=0;
8

```

```

9      L=22;
10     u1=yellowv;
11     u2=yellowv;
12     e2=e3;
13     e3=SetDataOne-dataOne;
14
15     u1=u2;
16     v[i]=u2;
17
18     b=float(0.9847);
19     c=float(0.9355);
20     a=float(15.3614);
21
22     i=i+1;
23     if(i<=L)
24         u2=(a*e3)-(a*b*e2)+c*u1;
25     else
26         u2=(a*e3)-(a*b*e2)+c*u1+(1-c)*v[i-L-1];
27
28     if(u2>20) u2=20;
29     if(u2<4) u2=4;
30
31     if(u2>yellow_backup)
32         dmconce=dmconce+1;
33     if(dmconce==1)
34         limit=u2;
35     if(u2<limit && dmconce<=40)
36         u2=limit;
37
38     yellowv=u2;
39     return u2;
40 }

```

D 卡尔曼控制器代码

```

1 float CMyView::dmcontrol1(float dataOne, float SetDataOne, float
   inDataTwo, float SetDataTwo, double Kp, double Ki, double Kd)
2 {
3     static float e2=0, e3=0;

```

```

4      static float u1=4, u2=4, eu=0;
5      static float v[65535];
6      static int L=0, i=0;
7      static float a=0, b=0, c=0;
8
9      L=22;
10     u1=yellowv;
11     u2=yellowv;
12     e2=e3;
13     e3=SetDataOne-dataOne;
14
15     u1=u2;
16     v[i]=u2;
17
18     b=float(0.9849);
19     c=float(0);
20     a=float(238.2);
21
22     i=i+1;
23     if(i<=L)
24         u2=(a*e3)-(a*b*e2)+c*u1;
25     else
26         u2=(a*e3)-(a*b*e2)+c*u1+(1-c)*v[i-L-1];
27
28     if(u2>20) u2=20;
29     if(u2<4) u2=4;
30
31     if(u2>yellow_backup)
32         dmconce=dmconce+1;
33     if(dmconce==1)
34         limit=u2;
35     if(u2<limit && dmconce<=40)
36         u2=limit;
37
38     yellowv=u2;
39     return u2;
40 }

```

E IMC 控制器代码

```

1 float CMyView::othercontrol1(float dataOne, float SetDataOne, float
  inDataTwo, float SetDataTwo, int kp, double ki, double kd)
2 {
3     static float e_imc = 0, e_imc_last = 0;
4     static float u1 = 4.0f, u2 = 4.0f;
5     static float y_model_dev = 0, y_model_dev_last = 0;
6     static float v[65535];
7     static int i = 0;
8
9     int L = 22;                                // 纯滞后步数: 22s / 1s = 22
10    float lambda = 15.0f;                       // 核心整定参数 (可直接修改)
11    float alpha = exp(-1.0f / lambda);           //  $\alpha = e^{-T/\lambda}$ 
12    float a1 = 0.9847f;                         // 离散惯性系数  $\exp(-1/65)$ 
13    float b1 = (1.0f - a1) * 0.275f;           // 增益系数  $K*(1-a1)$  0.004198
14
15    // 内部模型液位预测
16    float delta_u_delay = 0.0f;
17    if (i > L) {
18        delta_u_delay = v[i - L - 1] - 4.0f;
19    }
20    // 递推当前时刻免受外部干扰的模型理论液位增量
21    y_model_dev = a1 * y_model_dev_last + b1 * delta_u_delay;
22
23    // 内模误差反馈计算
24    // 设定增量 - (实际测得液位增量 - 模型理论预测增量)
25    e_imc = (SetDataOne - dataOne) + y_model_dev;
26
27    // 差分递推求解
28    float u_dev_last = u1 - 4.0f;               // 上一拍电流增量
29    // 求解当前拍所需的控制量, 并叠加 4mA 基准电流恢复为绝对物理输出
30    u2 = alpha * u_dev_last
31        + ((1.0f - alpha) / b1) * e_imc
32        - (((1.0f - alpha) * a1) / b1) * e_imc_last
33        + 4.0f;
34
35    y_model_dev_last = y_model_dev;
36    v[i] = u2;
37    i = i + 1;
38
39    // 物理上限幅约束及执行机构保护逻辑

```

```
40     if (u2 > (float)m_mvuplimit1 / 1000.0f) u2 = (float)m_mvuplimit1 /  
1000.0f;  
41     if (u2 < (float)m_mvdownlimit1 / 1000.0f) u2 = (float)  
m_mvdownlimit1 / 1000.0f;  
42  
43     if (u2 > yellow_backup) {  
44         dmconce = dmconce + 1;  
45         if (dmconce == 1) limit = u2;  
46     }  
47     if (u2 < limit && dmconce <= 40) u2 = limit;  
48  
49     yellowv = u2;  
50     return u2;  
51 }
```